

Cahier des charges — SecureAuth : coffre-fort de documents classifiés

Oct 5, 2026 · Khalil Hnich, Housseem Ben Khamou

Page de garde

[Logo de l'université TEK-UP : à insérer]

Université TEK-UP

Module **Cybersécurité** — Projet 2 : *Système sécurisé d'authentification et de cryptographie*

SecureAuth — Coffre-fort de documents classifiés

Cahier des charges

Réalisé par	Khalil Hnich · Housseem Ben Khamou
Encadrant	Oussema Bakali
Année universitaire	2026–2027
Version	1.0 — octobre 2026

1. Contexte et présentation du projet

SecureAuth est une application Web qui protège les documents classifiés d'une agence fictive : elle décide côté serveur qui peut lire quel document, chiffre chaque document et trace chaque accès dans un journal infalsifiable.

Le projet répond au sujet n°2 du module Cybersécurité (TEK-UP), réalisé en 6 semaines. Le sujet impose une application avec rôles utilisateur et administrateur, des sessions sécurisées, le hachage des mots de passe, une autorisation côté serveur, ainsi que du chiffrement et une démonstration HMAC ou signature.

L'authentification n'est que la porte d'entrée. Le cœur du projet est le contrôle d'accès aux documents et la preuve qu'il est respecté.

Analogie. L'agence est un bâtiment sécurisé :

- le **badge** (authentification) fait entrer dans le bâtiment ;

- l'**habilitation** dit à quels étages on peut monter ;
- le **besoin d'en connaître** dit quelles pièces de cet étage on peut ouvrir ;
- chaque document est dans un **coffre scellé** (chiffrement et sceau d'intégrité) ;
- chaque passage devant un garde est inscrit dans un **registre infalsifiable** (journal HMAC chaîné).

2. Objectifs et critères de réussite

Le projet vise cinq objectifs, chacun lié à une exigence du sujet et vérifié par une preuve technique.

#	Objectif	Exigence du sujet couverte
O1	Offrir deux rôles distincts : agent et administrateur	Application Web avec rôles utilisateur / administrateur
O2	Gérer inscription, connexion, déconnexion et sessions sécurisées	Inscription, connexion, déconnexion et sessions sécurisées
O3	Stocker les mots de passe hachés et contrôler les comptes	Hachage des mots de passe et contrôle des comptes
O4	Appliquer côté serveur les règles d'accès aux documents	Autorisation côté serveur et contrôle d'accès
O5	Chiffrer les documents et sceller le journal par HMAC	Chiffrement + démonstration HMAC ou signature

Critère de réussite. La solution est fonctionnelle, testée, documentée et présentée avec des preuves techniques. Règle du module : *un résultat de sécurité doit être démontré par des preuves.*

En pratique, chaque fonctionnalité de sécurité est acceptée lorsqu'elle a :

- au moins un test automatisé qui passe ;
- une vérification manuelle décrite, avec le résultat attendu ;
- une preuve exploitable dans le rapport (capture, extrait de journal, sortie de commande).

3. Périmètre

Le périmètre couvre des documents texte protégés par un contrôle d'accès à deux critères ; tout ce qui suit la ligne « hors périmètre » est cité dans le rapport comme perspective, sans être implémenté.

Inclus

- Comptes, rôles, inscription, connexion, déconnexion, changement de mot de passe.
- Sessions côté serveur, verrouillage après échecs, limitation de débit.
- Création, lecture et partage de documents texte classifiés.
- Espace administrateur : comptes, habilitations, activation, journal d'audit.
- Chiffrement des documents, journal signé par HMAC, transport TLS.
- Tests de sécurité automatisés et manuels (Burp Suite, OpenSSL, outils développeur).

Hors périmètre

- Téléversement de fichiers (texte uniquement).
- Authentification multifacteur.
- Règle *no write down* de Bell-LaPadula (présentée en théorie seulement).
- Déclassement ou reclassification des documents.
- Rotation des clés, HSM, coffre-fort de secrets.
- Migrations de schéma (Alembic) et API REST publique.

4. Acteurs et niveaux de classification

Trois acteurs interviennent ; l'administrateur gère le système mais son rôle ne lui donne accès à aucun document.

Acteur	Rôle technique	Peut	Ne peut pas
Agent	user	Créer des documents jusqu'à son habilitation, lire ceux auxquels il a droit, partager les siens	Lire au-dessus de son habilitation, lire hors liste d'accès, accéder à l'espace admin
Administrateur	admin	Gérer comptes, habilitations, activation et listes d'accès ; consulter et vérifier le journal	Lire un document par son rôle, s'attribuer un accès, modifier ses propres droits

Acteur	Rôle technique	Peut	Ne peut pas
Propriétaire	agent créateur d'un document	Accorder et retirer l'accès à son document	Perdre son propre accès, partager vers un agent insuffisamment habilité

Chaque document porte une **classification** ; chaque utilisateur porte une **habilitation** sur la même échelle.

Niveau	Libellé	Couleur du marquage
0	PUBLIC	vert
1	CONFIDENTIEL	bleu
2	SECRET	rouge
3	TRÈS SECRET	orange

Un nouvel inscrit est un agent de niveau 0. Le premier administrateur est créé uniquement en ligne de commande.

5. Exigences fonctionnelles

L'application expose 21 fonctions réparties en cinq domaines ; chaque exigence porte un identifiant pour être suivie jusqu'aux tests.

ID	Domaine	Exigence	Accès
EF-01	Authentification	S'inscrire avec nom d'utilisateur, e-mail et mot de passe	anonyme
EF-02	Authentification	Se connecter par nom d'utilisateur ou e-mail	anonyme
EF-03	Authentification	Se déconnecter (requête POST avec jeton CSRF)	connecté
EF-04	Authentification	Changer son mot de passe en fournissant l'actuel	connecté
EF-05	Authentification	Créer le premier administrateur en ligne de commande	serveur
EF-06	Comptes	Verrouiller un compte 15 min après 5 échecs consécutifs	système

ID	Domaine	Exigence	Accès
EF-07	Documents	Lister uniquement les documents lisibles	connecté
EF-08	Documents	Créer un document de niveau inférieur ou égal à son habilitation	connecté
EF-09	Documents	Lire un document autorisé (titre et contenu déchiffrés)	règle <code>can_read</code>
EF-10	Documents	Afficher la classification de chaque document par un bandeau coloré	connecté
EF-11	Partage	Consulter la liste d'accès d'un document	propriétaire ou admin
EF-12	Partage	Ajouter un agent à la liste d'accès	règle <code>can_grant</code>
EF-13	Partage	Retirer un agent de la liste d'accès	règle <code>can_revoke</code>
EF-14	Administration	Lister les comptes	admin
EF-15	Administration	Modifier l'habilitation d'un compte	règle <code>can_admin_modify</code>
EF-16	Administration	Activer ou désactiver un compte	règle <code>can_admin_modify</code>
EF-17	Administration	Voir les métadonnées d'un document (référence, classification, propriétaire), jamais son titre ni son contenu	admin
EF-18	Audit	Consulter le journal d'audit paginé	admin
EF-19	Audit	Vérifier l'intégrité du journal (page admin et commande <code>flask verify-audit</code>)	admin / serveur
EF-20	Audit	Journaliser les 17 types d'événements de sécurité, sans donnée sensible	système
EF-21	Démonstration	Générer le jeu de données de démo (<code>flask seed-demo</code>)	serveur

6. Exigences de sécurité

Toutes les décisions d'accès passent par un point de décision unique, le module `security/policy.py` ; aucune route ne vérifie un droit à la main.

6.1 Principes

1. **Pas de lecture vers le haut** (modèle Bell-LaPadula) : on ne lit jamais un document dont la classification dépasse son habilitation.
2. **Besoin d'en connaître** : l'habilitation ne suffit pas, il faut aussi figurer sur la liste d'accès du document.
3. **Séparation des rôles** : l'administrateur gère mais ne lit pas ; il ne s'attribue rien et ne modifie pas ses propres droits.

6.2 Règles d'autorisation

```
can_read(user, doc) =
    user.is_active
    AND NOT user.is_locked()
    AND user.clearance_level >= doc.classification
    AND exists DocumentAccess(document=doc, user=user)

can_create(user, level) =
    user.is_active AND level <= user.clearance_level

can_grant(actor, grantee, doc) =
    (actor is doc.owner OR actor.role == "admin")
    AND grantee.is_active
    AND grantee.clearance_level >= doc.classification
    AND NOT (actor.role == "admin" AND grantee == actor)

can_revoke(actor, grantee, doc) =
    (actor is doc.owner OR actor.role == "admin")
    AND grantee != doc.owner

can_admin_modify(actor, target) =
    actor.role == "admin" AND target != actor
```

6.3 Exigences complémentaires

ID	Exigence
ES-01	Document non autorisé ⇒ HTTP 404, jamais 403 : on ne confirme pas qu'un document classifié existe. Le refus est journalisé (<code>DOC_ACCESS_DENIED</code>).
ES-02	Espace admin non autorisé ⇒ HTTP 403 (son existence n'est pas secrète).
ES-03	L'utilisateur est rechargé depuis la base à chaque requête : désactivation et baisse d'habilitation prennent effet immédiatement.
ES-04	Les URL utilisent un identifiant public UUID4, jamais l'identifiant entier (défense en profondeur, pas contrôle d'accès).
ES-05	Message unique « Identifiants invalides » pour tout échec de connexion, avec un temps de réponse identique.
ES-06	Cookie de session <code>__Host-sid</code> : <code>Secure</code> , <code>HttpOnly</code> , <code>SameSite=Lax</code> , expiration après 30 min d'inactivité et 8 h au total.
ES-07	Identifiant de session régénéré à la connexion et au changement de mot de passe (anti-fixation).
ES-08	Sessions stockées côté serveur (Redis) : la déconnexion les détruit réellement.
ES-09	Jeton CSRF sur tous les formulaires POST.
ES-10	Politique de contenu stricte (CSP) : aucun script ni style inline.
ES-11	Redirection après connexion limitée aux chemins internes (anti-redirection ouverte).
ES-12	Jamais journalisés : mot de passe, contenu de document, identifiant de session complet, clé.

7. Exigences cryptographiques

Quatre mécanismes standards protègent les mots de passe, les documents, le journal et le transport ; aucun algorithme maison n'est autorisé (bibliothèques `argon2-cffi` et `cryptography` uniquement).

Besoin	Mécanisme	Exigence détaillée
Mots de passe	Argon2id	Paramètres par défaut (64 Mo, 3 passes, 4 fils) ; nouveau hachage si les paramètres sont renforcés ; hash factice pour un compte inexistant ; 12 à 128 caractères, différent du nom, absent d'une liste de mots de passe courants
Documents	AES-256-GCM	Nonce aléatoire de 12 octets à chaque chiffrement ; titre et contenu chiffrés ensemble ; données associées (AAD) = <code>public_id classification owner_id</code>
Journal d'audit	HMAC-SHA256 chaîné	Chaque ligne scelle la précédente ; une tête de chaîne détecte la suppression des dernières lignes ; écriture sous verrou <code>SELECT ... FOR UPDATE</code>
Transport	TLS 1.2 / 1.3	Certificat signé par une CA locale (OpenSSL) ; redirection 301 de HTTP vers HTTPS ; HSTS activé pour la démonstration

Pourquoi des données associées. Elles ne sont pas chiffrées mais sont scellées : si un attaquant passe un document de TRÈS SECRET à PUBLIC directement en base, le déchiffrement échoue (`InvalidTag`) et l'incident est journalisé.

Le tag d'une ligne du journal se calcule ainsi, la première ligne partant de 64 zéros :

```
\text{tag}_n = \mathrm{HMAC}\text{-}SHA256\left(K_{\mathrm{HMAC}}, \backslash\text{tag}_{n-1} \backslash, \backslash, \backslash, \mathrm{JSON}_{\mathrm{canonique}}\right)
```

Gestion des clés. `ENCRYPTION_KEY` et `HMAC_KEY` font 256 bits, sont distinctes, vivent dans `.env` hors de la base et du dépôt Git. L'application refuse de démarrer si l'une manque, est mal formée ou si les deux sont identiques.

Option. Un export de document signé Ed25519, vérifiable avec `openssl pkeyutl -verify`, illustre la différence entre HMAC (clé partagée) et signature (clé publique).

8. Exigences non fonctionnelles

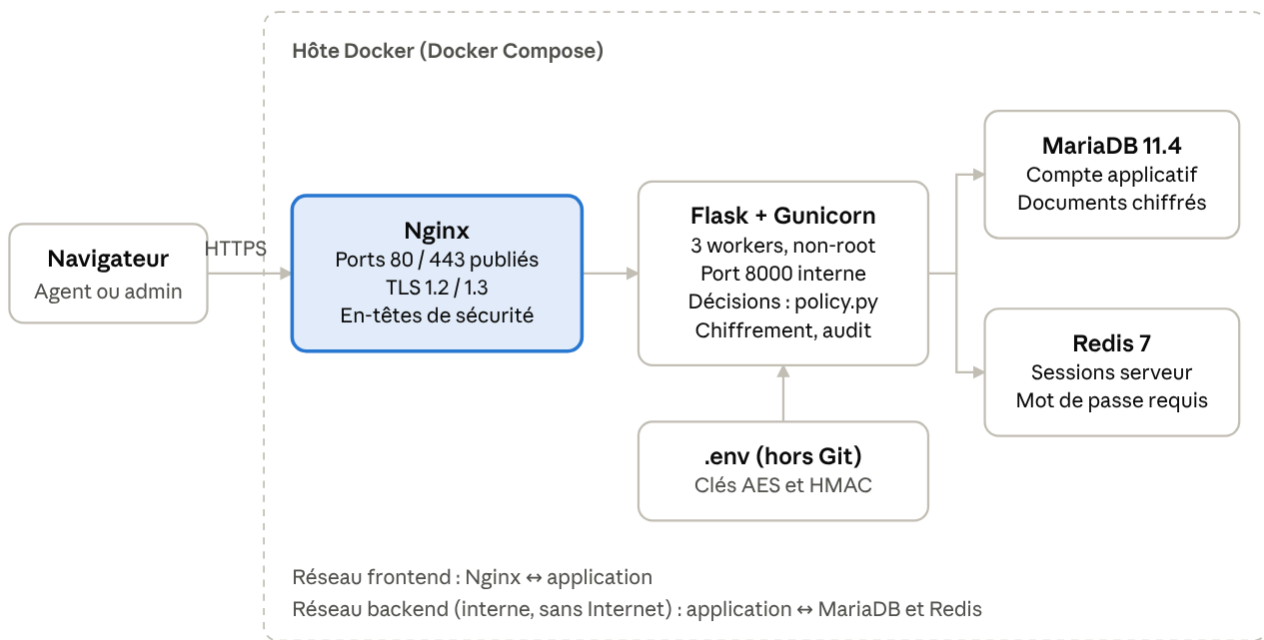
Les exigences non fonctionnelles fixent la qualité attendue sans affaiblir la sécurité.

ID	Catégorie	Exigence
ENF-01	Performance	Une vérification Argon2id prend environ 50 ms, identique pour un compte existant ou inconnu.
ENF-02	Concurrence	Le journal reste cohérent avec 3 workers Gunicorn écrivant en parallèle (aucune collision de chaîne).
ENF-03	Disponibilité	Chaque service a un contrôle de santé ; <code>/health</code> indique l'état de la base et de Redis sans jamais détailler une erreur.
ENF-04	Ergonomie	Interface sobre en français, classification visible par un bandeau coloré, affichage adapté au mobile.
ENF-05	Portabilité	Déploiement par Docker Compose ; fonctionne sous Windows (PowerShell, Git Bash) et Linux.
ENF-06	Maintenabilité	Code en anglais, commentaires et messages en français ; décisions d'accès centralisées dans un seul module.
ENF-07	Testabilité	Tests automatisés exécutables sans conteneur (SQLite en mémoire, Redis simulé), sans secret réel.
ENF-08	Moindre privilège	L'application tourne en utilisateur non-root, avec un compte de base limité à sa base et sans le mot de passe root.
ENF-09	Confidentialité des secrets	<code>.env</code> , la PKI et les certificats ne sont jamais commités.

9. Architecture technique et contraintes

Le navigateur ne parle qu'à Nginx ; l'application, MariaDB et Redis tournent dans des conteneurs que la machine hôte ne peut pas joindre directement.

Seul Nginx est exposé ; la base et Redis restent sur un réseau interne



architecture de déploiement · 4 conteneurs, 2 réseaux

Nginx termine le TLS et transmet à Flask sur le réseau frontend ; seule l'application rejoint le réseau backend, coupé d'Internet. Les secrets sont injectés uniquement là où ils servent : l'application ne reçoit jamais le mot de passe root de la base.

Contrainte	Choix imposé ou retenu
Langage et framework	Python 3.12, Flask 3.1
Serveur d'application	Gunicorn, 3 workers, derrière Nginx 1.27
Base de données	MariaDB 11.4 via l'ORM SQLAlchemy
Sessions	Redis 7 via Flask-Session
Formulaires et CSRF	Flask-WTF
Cryptographie	argon2-cffi, cryptography
Déploiement	Docker Compose, 4 services
Outils de test	pytest, Burp Suite, OpenSSL, outils développeur du navigateur

10. Modèle de données

La base MariaDB compte cinq tables ; seule la référence d'un document reste en clair, son titre et son contenu sont chiffrés.

Table	Rôle	Colonnes clés
<code>users</code>	Comptes et habilitations	<code>username</code> (unique, 3-32 car.), <code>email</code> (unique), <code>password_hash</code> (Argon2id), <code>role</code> , <code>clearance_level</code> (0-3), <code>is_active</code> , <code>failed_attempts</code> , <code>locked_until</code>
<code>secure_documents</code>	Documents chiffrés	<code>public_id</code> (UUID4), <code>reference</code> (en clair, ex. TS-2026-0007), <code>classification</code> (0-3), <code>owner_id</code> , <code>nonce</code> (12 octets), <code>ciphertext</code> (titre + contenu + tag GCM)
<code>document_access</code>	Listes d'accès (besoin d'en connaître)	<code>document_id</code> , <code>user_id</code> , <code>granted_by</code> , <code>granted_at</code> ; couple (<code>document_id</code> , <code>user_id</code>) unique
<code>audit_logs</code>	Journal signé	<code>created_at</code> (précision microseconde), <code>actor_id</code> , <code>event</code> , <code>outcome</code> , <code>target_type</code> , <code>target_id</code> , <code>ip_address</code> , <code>details</code> , <code>prev_tag</code> , <code>integrity_tag</code>
<code>audit_chain_head</code>	Tête de chaîne (une ligne)	<code>last_log_id</code> , <code>last_tag</code>

Relations. Un utilisateur possède des documents ; un document a une liste d'accès de plusieurs utilisateurs ; chaque ligne d'audit référence au plus un acteur.

Choix notables.

- Le titre est chiffré : le titre d'un document secret est lui-même une information sensible.
- `audit_logs.created_at` stocke les microsecondes, sinon la date relue différerait de la date signée et la chaîne serait déclarée corrompue.
- Un compte n'est jamais supprimé mais désactivé : supprimer l'acteur modifierait une ligne d'audit déjà signée.

Événements d'audit (17). `REGISTER`, `LOGIN_SUCCESS`, `LOGIN_FAILURE`, `LOGIN_LOCKED`, `LOGOUT`, `PASSWORD_CHANGE`, `DOC_CREATE`, `DOC_READ`, `DOC_ACCESS_DENIED`, `ACCESS_GRANT`, `ACCESS_REVOKE`, `CLEARANCE_CHANGE`, `ACCOUNT_DISABLE`,

ACCOUNT_ENABLE , ADMIN_SELF_MODIFY_DENIED , INTEGRITY_FAILURE , AUDIT_VERIFY .

11. Modèle de menace

Quatorze menaces sont retenues ; chacune a une contre-mesure et au moins un cas de test de la section 12.

Menace	Exemple d'attaque	Contre-mesure	Test
Énumération de comptes	Tester des e-mails sur le formulaire de connexion	Message unique ; hash factice pour un temps de réponse identique	T01
Force brute	Essais de mots de passe en masse	Argon2id ; verrouillage après 5 échecs ; limitation de débit	T02, T25
Vol de session	XSS, interception réseau	Cookie HttpOnly et Secure , TLS, expiration, CSP	T04, T07
Fixation de session	Imposer un identifiant avant connexion	Régénération de l'identifiant à la connexion	T05
CSRF	Formulaire piégé sur un autre site	Jeton CSRF + SameSite	T08
IDOR	Changer l'identifiant dans l'URL	can_read() systématique, réponse 404	T11
Lecture vers le haut	Agent SECRET vise un document TRÈS SECRET	Comparaison habilitation / classification	T09
Abus de l'administrateur	L'admin lit ou s'octroie un accès	Séparation des rôles, auto-modification interdite, tout journalisé	T12, T13
Accès direct à la base	Lire les documents dans MariaDB	AES-256-GCM, clé hors base	T18
Déclassement frauduleux	Passer un document de 3 à 0 en base	Classification scellée dans les données associées GCM	T17
Effacement de traces	Modifier ou supprimer des lignes du journal	Chaîne HMAC + tête de chaîne	T19, T20

Menace	Exemple d'attaque	Contre-mesure	Test
Injection SQL	' OR 1=1 --	ORM SQLAlchemy, aucune requête concaténée	T03
XSS stocké	Script dans un titre de document	Échappement Jinja2 + CSP	T21
Élévation par la base	Modifier <code>clearance_level</code> directement en base	Limite assumée ; option : tag HMAC sur les habilitations	—

12. Stratégie de test et critères de recette

La recette repose sur 25 cas de test exécutés en semaine 4 ; un cas en échec n'est pas corrigé sur place mais documenté comme vulnérabilité trouvée, puis corrigé et rejoué en semaine 5 (avant / après).

Chaque cas est consigné dans `docs/tests-securite.md` avec : identifiant, scénario, outil, résultat attendu, résultat obtenu, preuve (fichier dans `docs/preuves/`).

ID	Catégorie	Scénario	Outil	Résultat attendu
T01	Authentification	Compte inexistant vs mauvais mot de passe	Burp / pytest	Même message, temps comparable
T02	Authentification	6 échecs consécutifs	pytest	Compte verrouillé 15 min
T03	Authentification	Injection SQL dans le formulaire	Burp	Échec normal, aucune erreur SQL
T04	Session	Drapeaux du cookie	Outils développeur	<code>Secure</code> , <code>HttpOnly</code> , <code>SameSite=Lax</code> , préfixe <code>__Host-</code>
T05	Session	Identifiant avant / après connexion	Outils développeur	Identifiant différent
T06	Session	Rejouer un cookie après déconnexion	Burp Repeater	Redirection vers la connexion

ID	Catégorie	Scénario	Outil	Résultat attendu
T07	Session	Inactivité > 30 min	Manuel / pytest	Session expirée
T08	CSRF	POST sans jeton	Burp	HTTP 400
T09	Autorisation	Agent SECRET → document TRÈS SECRET	pytest / Burp	404 + DOC_ACCESS_DENIED
T10	Autorisation	Habilitation suffisante mais hors liste d'accès	pytest / Burp	404
T11	Autorisation	IDOR : public_id valide d'un autre document	Burp	404
T12	Autorisation	L'admin tente de lire un document	pytest	404
T13	Autorisation	L'admin s'attribue un accès ou modifie sa propre habilitation	pytest / Burp	Refus + ADMIN_SELF_MODIFY_DENIED
T14	Autorisation	Un agent appelle une route /admin/*	Burp	403
T15	Autorisation	Habilitation abaissée après attribution	pytest	Accès perdu immédiatement

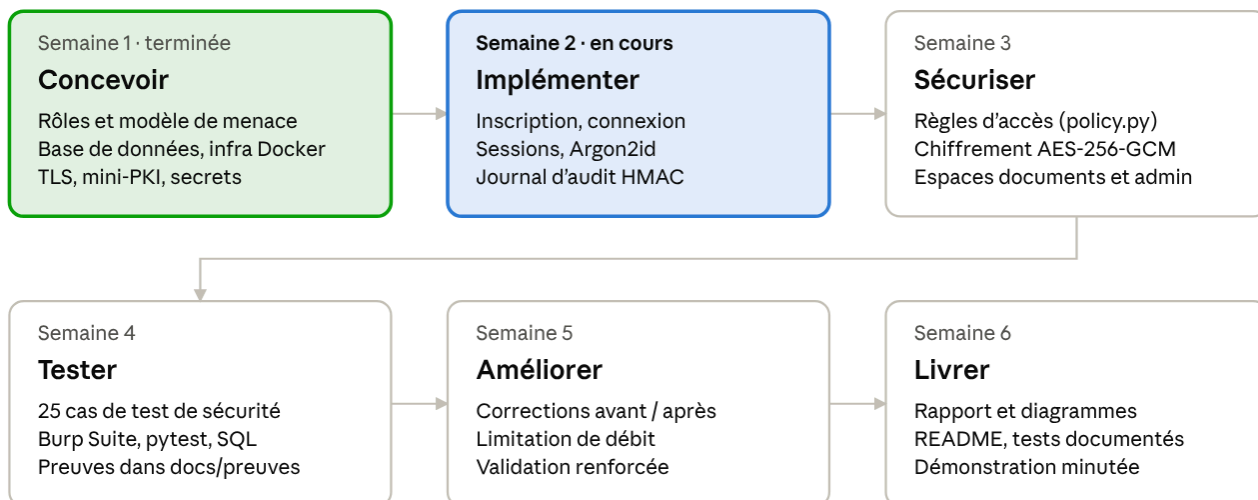
ID	Catégorie	Scénario	Outil	Résultat attendu
T16	Autorisation	Création au-dessus de son habilitation (champ modifié)	Burp	Refus
T17	Cryptographie	Classification modifiée directement en base	SQL + application	Déchiffrement impossible + <code>INTEGRITY_FAILURE</code>
T18	Cryptographie	Lecture des documents directement en base	SQL	Uniquement du binaire illisible
T19	Intégrité	Ligne du journal modifiée en base	SQL + <code>verify-audit</code>	Ligne signalée
T20	Intégrité	Ligne supprimée au milieu ou à la fin	SQL + <code>verify-audit</code>	Rupture détectée
T21	XSS	<code><script></code> dans un titre de document	Navigateur	Affiché comme texte, bloqué par la CSP
T22	Transport	Connexion en TLS 1.1	<code>openssl s_client -tls1_1</code>	Refusée
T23	Transport	En-têtes de sécurité	<code>curl -I</code>	Tous présents
T24	Infrastructure	Accès direct aux ports 3306 / 6379	<code>docker compose ps , Test-NetConnection</code>	Injoignables
T25	Force brute	Rafale de connexions	Burp Intruder	HTTP 429 après le seuil

Critère d'acceptation global. Les 25 cas obtiennent le résultat attendu après la semaine 5, et la démonstration finale (section 13) se déroule sans écart.

13. Planning et livrables

Le projet suit les six phases imposées par le sujet, une par semaine ; l'infrastructure de la semaine 1 est terminée et l'authentification de la semaine 2 est en cours.

Six semaines, de la conception à la démonstration ; la semaine 2 est en cours



planning · 6 semaines, 6 phases

Une phase est close lorsque chacune de ses tâches a son test automatisé, sa vérification manuelle et sa preuve.

Livrables finaux

- Code source et base de données (dépôt Git, `docker-compose.yml`, scripts d'initialisation).
- Diagrammes : architecture, composants, modèle de données, séquences de connexion et de lecture, cas d'utilisation.
- Dossier de tests de sécurité `docs/tests-securite.md` avec ses preuves.
- Rapport : architecture, méthodologie, résultats, risques, décisions, remédiations, limites.
- Démonstration d'environ 5 minutes.

Scénario de démonstration

1. Infrastructure : conteneurs actifs, cadenas TLS, en-têtes de sécurité.
2. alice (TRÈS SECRET) se connecte et voit TS-001, SEC-001 et PUB-001.

3. bob (SECRET) ne voit pas TS-001 ; forcer son URL renvoie 404 et le refus apparaît dans le journal.
4. alice tente de partager TS-001 avec bob : refusé, habilitation insuffisante.
5. L'administrateur baisse l'habilitation de bob à 1 : bob perd immédiatement SEC-001 et SEC-002.
6. L'administrateur tente d'ouvrir TS-001 : 404, il gère mais ne lit pas.
7. Attaque en base : TS-001 passé à PUBLIC, le déchiffrement est refusé (`INTEGRITY_FAILURE`).
8. Attaque sur le journal : une ligne modifiée est désignée par `verify-audit`.

14. Risques, limites assumées et perspectives

Les limites ci-dessous sont connues et assumées ; elles seront présentées dans le rapport avec leur atténuation.

Limite ou risque	Conséquence	Atténuation
Modification directe de <code>clearance_level</code> en base	Un attaquant ayant accès à la base s'élève sans être détecté	Base isolée sur un réseau interne ; option : tag HMAC sur les lignes d'habilitation
Possession de <code>HMAC_KEY</code>	Celui qui détient la clé peut recalculer une chaîne falsifiée valide	Clé hors base et hors dépôt, jamais transmise au conteneur de base
Verrouillage détourné	5 mauvais essais suffisent à bloquer le compte d'un autre pendant 15 min	Limitation de débit par IP (Flask-Limiter + nginx)
Collusion administrateur + complice	L'admin peut élever un complice et lui donner accès	Action journalisée et vérifiable ; non empêchée techniquement
CA locale sans CRL ni OCSP	Un certificat compromis ne peut pas être révoqué	Usage limité au développement ; <code>--ssl-no-revoke</code> sous Windows
Régénération de <code>.env</code>	Les documents déjà chiffrés deviennent illisibles	Procédure documentée ; clé inchangée en développement
Conflit de ports avec XAMPP	Apache occupe 80/443 et masque nginx	Service Apache passé en démarrage manuel

Perspectives. Authentification multifacteur, rotation des clés et stockage dans un HSM ou un coffre-fort de secrets, règle *no write down*, reclassification contrôlée des documents.

15. Glossaire

Terme	Définition
AAD	Données associées : informations non chiffrées mais scellées par AES-GCM ; leur modification fait échouer le déchiffrement.
AES-256-GCM	Chiffrement symétrique authentifié : il garantit à la fois la confidentialité et l'intégrité.
Argon2id	Fonction de hachage de mots de passe, volontairement lente et gourmande en mémoire contre la force brute.
Bell-LaPadula	Modèle de confidentialité : pas de lecture vers le haut, pas d'écriture vers le bas.
Besoin d'en connaître	Principe selon lequel l'habilitation ne suffit pas : il faut une autorisation explicite par document.
CSP	Content Security Policy : en-tête qui limite les sources de scripts et de styles autorisées dans le navigateur.
CSRF	Attaque qui fait exécuter une action à un utilisateur connecté depuis un autre site.
Fixation de session	Attaque qui impose à la victime un identifiant de session connu de l'attaquant.
Habilitation	Niveau maximal de classification qu'un utilisateur est autorisé à lire.
HMAC	Code d'authentification de message à clé secrète, qui prouve l'intégrité d'une donnée.
IDOR	Accès à un objet d'un autre utilisateur en changeant son identifiant dans la requête.
Nonce	Valeur aléatoire utilisée une seule fois par chiffrement.

Terme	Définition
TLS	Protocole qui chiffre les échanges entre le navigateur et le serveur.
XSS	Injection de script exécuté dans le navigateur d'une autre personne.